Freshness Classification of Korean Food

Digital Image Processing

2026

Contents

1. Datasets

1.1. Available Datasets

Seven publicly available datasets were identified and surveyed. Table 1 summarizes their properties. Together they cover fruits, vegetables, and citrus varieties, with fresh/rotten binary labels and, in one case, a three-class annotation including formalin-mixed samples. One dataset (Ripening Stages) uses an object detection format (YOLOv5) rather than image classification and is therefore excluded from training experiments.

Table 1: Datasets surveyed. †Object detection format — excluded from classification experiments.

Dataset	Categories	Classes	Images	Size	Source
FruitVision	Apple, Banana, Grape, Mango, Orange	Fresh / Formalin / Rotten	10,154	857 MB	Mendeley
Lemon Varieties	7 lemon varieties	Fresh / Rotten	1,956	1.35 GB	Mendeley
Ripening Stages†	Strawberry, Avocado	Unripe / Partial / Ripe / Rotten	—	1.97 GB	Mendeley
Fruits Classification	Peach, Pomegranate, Strawberry	Fresh / Rotten	1,655	14.5 MB	Mendeley
Fruits Quality	12 mixed fruits	Fresh / Rotten	359	12.4 MB	Kaggle
Fruits Fresh/Rotten	Apple, Banana, Orange	Fresh / Rotten	13,599	1.95 GB	Kaggle
Fresh & Stale	Apple, Banana, Cucumber, Okra, Orange, Potato, Tomato	Fresh / Rotten	27,317	3.09 GB	Kaggle
Total (classification datasets)			**55,040**		

1.2. Dataset Sources

- **FruitVision** — `https://data.mendeley.com/datasets/xkbjx8959c/2`
- **Lemon Varieties** — `https://data.mendeley.com/datasets/mygrsk3vyb/2`
- **Ripening Stages** — `https://data.mendeley.com/datasets/mygrsk3vyb/2`
- **Fruits Classification** — `https://data.mendeley.com/datasets/rg254yr63x/1`
- **Fruits Quality** — `https://www.kaggle.com/datasets/nourabdoun/fruits-quality-fresh-v`

- **Fruits Fresh/Rotten** — `https://www.kaggle.com/datasets/sriramr/fruits-fresh-and-ro`
- **Fresh & Stale** — `https://www.kaggle.com/datasets/swoyam2609/fresh-and-stale-classi`

1.3. Korean Food Dataset — Proposal

No labeled dataset for Korean food freshness currently exists in public repositories. Table 2 lists the target food categories and classification states for the proposed dataset.

Table 2: Target categories for the proposed Korean food freshness dataset.

Category	Romanization	States	Notes
Kimchi	kimchi	Fresh / Rotten	Color and mold during spoilage
Rice cake	tteok	Fresh / Rotten	Mold visible on surface
Side dishes	banchan	Fresh / Rotten	Vegetable-based, wilting
Cooked rice	bap	Fresh / Rotten	Mold indicator
Mandarin (Jeju)	gyul	Fresh / Semifresh / Rotten	Jeju variety, citrus
Strawberry	ttalgi	Fresh / Rotten	Fast spoilage

2. Methodology

2.1. Overall Pipeline

The classification pipeline consists of four sequential stages:

Stage	Component	Description
1	Data preparation	Raw datasets reorganized into unified `food/state/` folder structure
2	Feature extraction	Pre-trained CNN backbone (ImageNet weights), frozen or partially unfrozen
3	Projection head	$d_{\text{out}} \to 256 \to 128$ linear projection with ReLU
4	Classification	Linear layer $128 \to C$ with cross-entropy loss

All input images are resized to 224×224 pixels. Training images undergo data augmentation (random resized crop, horizontal flip, $\pm 30°$ rotation, color jitter on brightness/contrast/saturation/hue, Gaussian blur) to increase robustness to lighting and viewpoint variation. Validation images are center-cropped without augmentation. All images are normalized with ImageNet channel statistics ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$).

2.2. Backbone Architectures

Six backbone architectures are considered. Table 3 summarizes their properties. Preliminary experiments are conducted with ResNet-18 as the primary backbone. EfficientNet-B0 is identified as the recommended architecture for the final Korean food classifier based on its accuracy-to-parameter ratio and demonstrated suitability for small datasets.

Table 3: Backbone architectures considered in this study.

Model	Params	Out dim	Family
ResNet-18	11.2M	512	ResNet
ResNet-34	21.3M	512	ResNet
ResNet-50	25.6M	2048	ResNet
MobileNetV3-S	2.5M	576	MobileNet
EfficientNet-B0	5.3M	1280	EfficientNet
EfficientNet-B2	9.1M	1408	EfficientNet

2.3. Training Conditions

Four training conditions form an ablation study to determine the optimal degree of backbone fine-tuning. Conditions C3 and C4 (with projection head) are the focus of this study, as the additional non-linear projection consistently improves performance over a direct linear probe.

Table 4: Training conditions (ablation study).

ID	Name	Backbone	Head	Description
C1	`frozen`	Frozen	Linear only	No backbone adaptation
C2	`layer4`	Layer4 free	Linear only	Partial fine-tuning, no projection
C3	`head_frozen`	Frozen	Projection + Linear	Full head, frozen backbone
C4	`head_layer4`	Layer4 free	Projection + Linear	Full head + partial fine-tuning

In all conditions the backbone is initialized with ImageNet pre-trained weights and produces a feature vector of dimension d_{out} (e.g. 512 for ResNet-18, Table 3).

C1 – frozen backbone, linear head. All backbone weights are frozen; gradients do not flow through the convolutional layers. A single linear layer $d_{\text{out}} \to C$ is trained from scratch. This is the classical *linear probe* setting, where the pre-trained backbone acts purely as a fixed feature extractor.

C2 – layer4 unfrozen, linear head. The last residual block of the backbone (`layer4` in ResNet notation, comprising two residual units totaling $\approx$2M parameters for ResNet-18) is unfrozen and fine-tuned jointly with the linear head. All earlier layers remain frozen. This allows the backbone to specialize its high-level features to the target domain without the cost of full fine-tuning.

C3 – frozen backbone, projection head. A two-layer non-linear projection head ($d_{\text{out}} \to 256 \to 128$ with ReLU) is appended before the final linear classifier. The backbone remains completely frozen; only the head is trained. The additional capacity of the projection layers allows the model to learn a more discriminative embedding space without modifying the backbone.

C4 – layer4 unfrozen, projection head. Combines the projection head of C3 with the partial fine-tuning of C2. The last backbone block (`layer4`) and the full projection head are optimized jointly, with a lower learning rate applied to the backbone ($\text{lr}_{\text{backbone}} = 10^{-5}$) to prevent catastrophic forgetting. This is the most expressive configuration and consistently achieves the best validation F1.

All models are trained with Adam optimizer ($\text{lr} = 10^{-3}$, $\text{lr}_{\text{backbone}} = 10^{-5}$, weight decay $= 10^{-4}$) and cosine annealing over 50 epochs. Batch size is 32.

2.4. Label Modes

Two classification targets are evaluated:

- **state**: classes correspond to freshness states only (fresh / rotten, or fresh / formalin / rotten). The model learns state-invariant features across all food categories.
- **fruit_state**: classes correspond to food-state combinations (e.g., `apple_fresh`, `apple_rotten`). The model simultaneously learns food identity and freshness state, enabling a richer confusion matrix analysis.

2.5. Evaluation Metrics

Given the potential class imbalance in real-world Korean food datasets, accuracy alone is insufficient. Let TP_c, FP_c, FN_c, and TN_c denote true positives, false positives, false negatives, and true negatives for class c, and let C be the total number of classes. The following metrics are reported:

- **Accuracy**: fraction of all samples correctly classified,

$$\text{Accuracy} = \frac{\sum_{c=1}^{C} TP_c}{\text{total samples}}. \tag{1}$$

- **Precision** (per class): fraction of predicted positives that are truly positive,

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}. \tag{2}$$

- **Recall** (per class): fraction of actual positives correctly identified,

$$\text{Recall}_c = \frac{TP_c}{TP_c + FN_c}. \tag{3}$$

- **F1-score** (per class): harmonic mean of precision and recall,

$$F1_c = \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c} = \frac{2\,TP_c}{2\,TP_c + FP_c + FN_c}. \tag{4}$$

- **Macro averages**: each per-class metric is averaged uniformly over all classes,

$$\overline{F1} = \frac{1}{C}\sum_{c=1}^{C} F1_c, \qquad \overline{\text{Precision}} = \frac{1}{C}\sum_{c=1}^{C} \text{Precision}_c, \qquad \overline{\text{Recall}} = \frac{1}{C}\sum_{c=1}^{C} \text{Recall}_c. \tag{5}$$

  Macro averaging treats every class equally regardless of sample count, which is essential when minority classes (e.g. formalin) are safety-critical.

- **Confusion matrix**: row-normalized per true class, to identify systematic misclassification patterns.

For food safety applications, recall on the *rotten* class is particularly critical: a false negative (rotten food classified as fresh) carries higher risk than a false positive.

3. Preliminary Experiments

3.1. Setup

All experiments in this section use ResNet-18 with ImageNet pre-trained weights. Conditions C3 and C4 (with projection head) are the primary focus. A stratified 80/20 train/validation split is applied to all datasets. Experiments marked *pending* are in progress at the time of this report.

3.2. Results

Table 5 reports the best macro F1-score achieved per dataset and condition combination.

Table 5: Best macro F1-score (%) — ResNet-18, conditions C3 and C4. *Pending*: experiment in progress. N/A: fruit_state not applicable (single-fruit dataset). Bold: best result per dataset.

Dataset	C3: head_frozen		C4: head_layer4	
	state	fruit_state	state	fruit_state
FruitVision (3-class)	82.6	86.2	**88.0**	*pending*
Fruits Classification	94.0	94.5	**95.8**	*pending*
Lemon Varieties	*pending*	N/A	**98.2**	N/A
Fruits Fresh/Rotten	99.3	99.2	**99.7**	99.5
Fresh & Stale (18 classes)	97.9	96.1	**99.0**	97.7
Fruits Quality	*pending*	N/A	*pending*	N/A

3.3. All Runs — Detailed Results

Table 6 lists every completed experiment sorted by accuracy. All runs use ResNet-18 with all available fruits. Rows marked † used an earlier code version that ran for only 30 epochs and did not record F1, precision, or recall; accuracy is included for reference only.

Table 6: All completed runs sorted by validation accuracy. All experiments: ResNet-18, all fruits. †Early-version run (30 epochs); F1/Prec/Recall not recorded.

Dataset	Condition	Acc	F1	Prec	Recall	Ep
Fruits Fresh/Rotten	head_layer4	99.7	99.7	99.7	99.7	50
Fruits Fresh/Rotten	head_frozen	99.3	99.3	99.3	99.3	50
Fresh & Stale	head_layer4	99.0	99.0	99.0	99.0	50
Lemon Varieties	head_layer4	98.2	98.2	98.2	98.2	50
Fresh & Stale	head_frozen	97.9	97.9	97.9	97.9	50
Fruits Classification	head_layer4	95.8	95.8	95.8	95.8	50
Fruits Classification	head_frozen	94.0	94.0	94.0	94.0	50
Fruits Classification†	frozen	93.4	—	—	—	30
FruitVision	head_layer4	87.9	88.0	88.7	87.7	50
FruitVision†	layer4	87.1	—	—	—	30
FruitVision	head_frozen	82.2	82.6	83.0	82.9	50
FruitVision†	frozen	75.3	—	—	—	30

3.4. Confusion Matrix Analysis

Figure 1 shows the row-normalized confusion matrices for two representative completed experiments.

3.5. Learning Curves

Figures 2–5 show the per-epoch metrics for all completed runs, grouped by dataset. Each curve corresponds to one training condition and backbone combination.

4. Korean Food Dataset Proposal

4.1. Data Collection Pipeline

Two automated image collection scripts have been implemented, one targeting Korean fruits and one targeting Korean prepared foods. Both use Bing Image Search with 5–8 search queries per class per freshness state, combining Korean-language and romanized search terms to maximize result diversity (e.g., *"fresh kimchi bright red color"* and *"moldy kimchi white fungus"* for the kimchi category).

Downloaded images are validated and processed as follows:

- **Integrity check**: corrupt or unreadable files are discarded.
- **Minimum resolution**: 100×100 pixels.
- **Format**: any format accepted; converted to RGB JPEG.
- **Output size**: 256×256 pixels (model crops to 224×224 during training).

4.2. Integration with Training Pipeline

The Korean food dataset will be organized in the same `food/state/` nested structure used throughout this study, making it directly compatible with the existing training and evaluation pipeline without any code modifications. The `prepare_datasets.py`

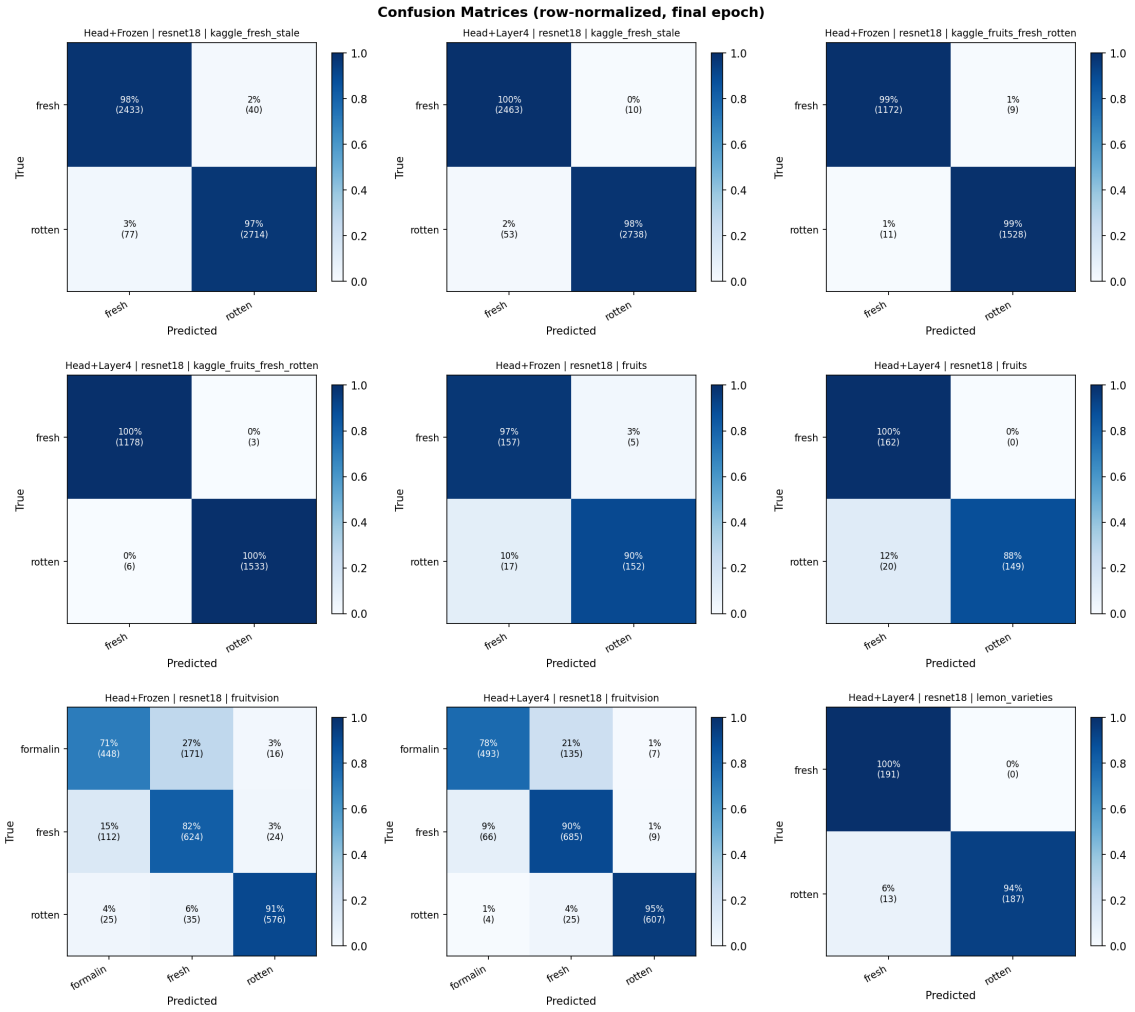

Figure 1: Row-normalized confusion matrices (final epoch). The formalin class in FruitVision shows the highest misclassification rate, predominantly confused with fresh — analogous to the challenge of detecting subtle spoilage in Korean fermented foods. The Kaggle Fruits Fresh/Rotten matrix (6 classes, fruit_state mode) shows near-perfect diagonal structure with essentially no cross-fruit confusion.

script can be extended with a new preparation function, and the training pipeline selects the dataset interactively at runtime.

5. Software Implementation

The classification system is implemented in Python (PyTorch) and organized into five modules. Figure 6 illustrates the data flow between components.

5.1. Usage Workflow

Only two scripts are executed directly by the user; the remaining modules are imported internally by `train.py`.

1. **`python prepare_datasets.py`** — Run once per dataset. Reorganizes the raw images from `datasets/` into the unified `data/{dataset}/{fruit}/{state}/` structure. Must be run before any training.

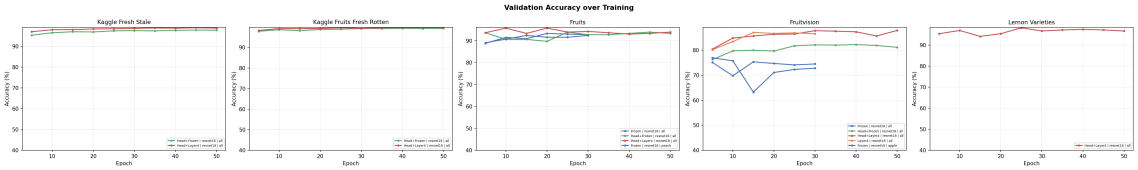

Figure 2: Validation accuracy over training epochs per dataset.

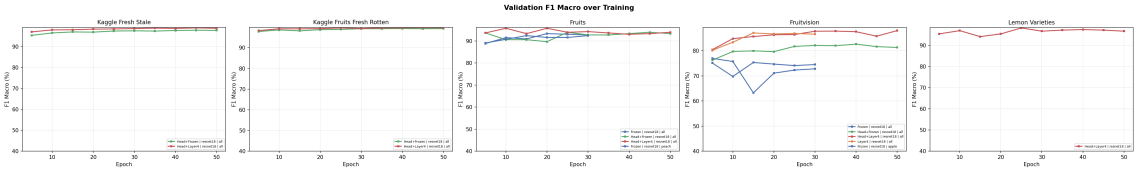

Figure 3: Macro-averaged F1 score over training epochs per dataset.

2. **`python train.py`** — Run once per experiment. Presents an interactive menu to select dataset, label mode, condition, and backbone. Internally imports `backbone.py` (to build the model) and `dataset.py` (to build the data loaders); these two modules are never executed directly. Saves results to `run_outputs/{run_name}/`.
3. **`python analyze.py`** — Run after one or more training experiments are complete. Reads all `metrics.json` files and produces comparison plots across all runs.
4. **`python docs/prepare_figures.py`** — Copies the generated plots into `docs/figures/` for inclusion in this report.

`backbone.py` and `dataset.py` are never run directly. They are libraries: `backbone.py` provides the backbone registry and `dataset.py` provides the data loaders. Both are imported at the top of `train.py` and called transparently during training.

5.2. `prepare_datasets.py` — Dataset Preparation

Raw datasets are downloaded from their respective sources and placed in the `datasets/` directory, each in its original folder structure. `prepare_datasets.py` reads each raw dataset and reorganizes it into a unified `data/{dataset}/{fruit}/{state}/` hierarchy compatible with PyTorch's `ImageFolder` loader. The script handles format differences across datasets: flat structures (all images in one folder), split structures (pre-divided `train/test/`), and nested structures (fruit then state sub-folders). Each dataset has a dedicated preparation function that normalizes class names, resolves naming inconsistencies (e.g. `freshpatato` → `potato/fresh`), and copies or hard-links images to the output directory. The script is idempotent: re-running it on an already-prepared dataset overwrites nothing.

5.3. `backbone.py` — Backbone Registry

A central registry (`BACKBONE_REGISTRY`) maps string identifiers (`"resnet18"`, `"efficientnet_b0"`, etc.) to factory functions that return pre-trained models loaded from `torchvision`. The `GenericBackbone` wrapper exposes a unified interface:

- `forward(x)` returns the feature vector of dimension d_{out} (the output of the

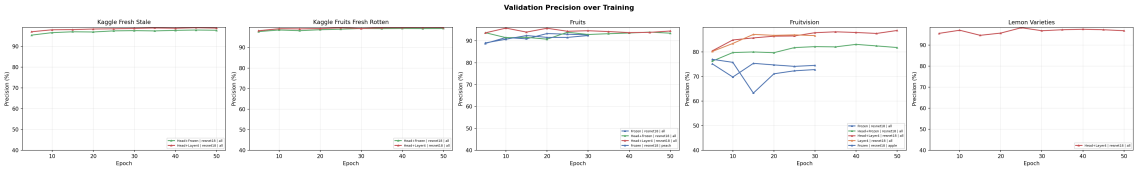

Figure 4: Macro-averaged precision over training epochs per dataset.

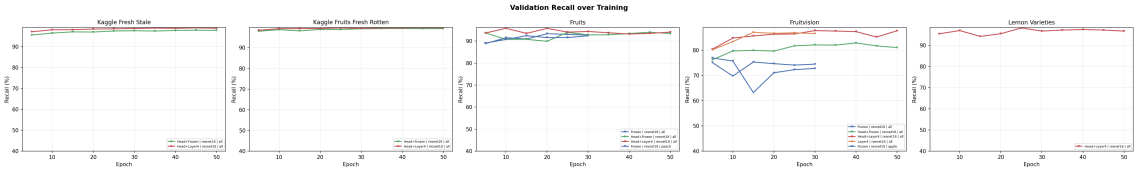

Figure 5: Macro-averaged recall over training epochs per dataset.

backbone's global pooling layer, *before* the original classification head).

- `unfreeze_layers(n)` selectively unfreezes the last n layer groups of the backbone, leaving all earlier weights frozen. For ResNet this corresponds to the last n residual stages (`layer4`, `layer3`, ...).

This design decouples the backbone choice from the training loop, making it trivial to swap architectures.

5.4. `dataset.py` — Data Loading

`dataset.py` provides three loaders that cover the three folder layouts found across the six datasets:

- **Layout A** (flat): all images in a single directory.
- **Layout B** (split): pre-divided `train/` and `test/` sub-folders.
- **Layout C** (nested): `fruit/state/` hierarchy, used for all experiments in this study.

For Layout C, `get_loaders_nested()` constructs a stratified 80/20 train/validation split using scikit-learn's `StratifiedShuffleSplit`. Two separate `ImageFolder` objects (one with augmentation transforms, one with validation-only transforms) are created over the same index sets so that training and validation images receive different preprocessing. The `label_mode` parameter controls whether class labels are derived from freshness state only (`state`) or from fruit-state combinations (`fruit_state`).

5.5. `train.py` — Training Loop

The training script presents an interactive menu for dataset, label mode, condition, and backbone selection. The run directory name is deterministically computed from these choices (e.g. `mendeley_fruitvision_all_head_layer4_resnet18_fs`), which serves as both a unique identifier and a resume key.

Training proceeds for a fixed number of epochs (default 50) with the following behavior:

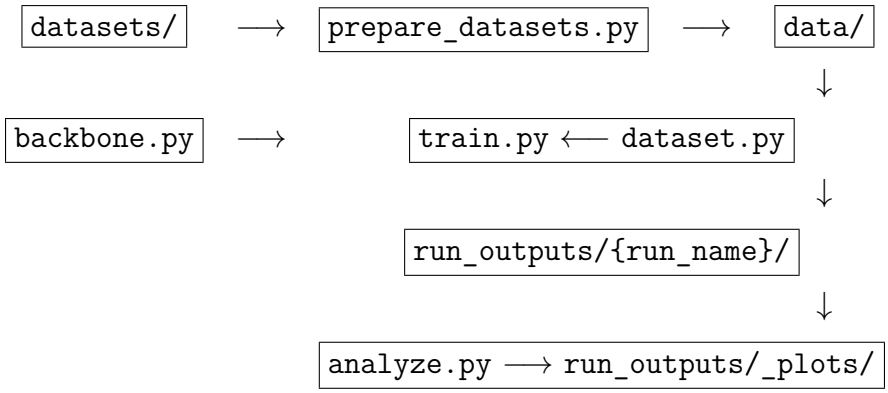

Figure 6: Data flow between the five software modules.

- **Evaluation every 5 epochs**: accuracy, macro F1, macro precision, macro recall, per-class metrics, and confusion matrix are computed on the validation set and appended to `metrics.json`.
- **Rolling checkpoint every 10 epochs**: a single `checkpoint.pt` file is overwritten, keeping disk usage constant.
- **Best model**: whenever validation F1 improves, the full model state is saved to `best_model.pt`.
- **Resume**: if a checkpoint exists in the run directory, training automatically resumes from the last saved epoch.

At the end of training, all metrics are serialized to `metrics.json`, which serves as the single source of truth for all subsequent analysis.

5.6. `analyze.py` — Visualization

`analyze.py` scans `run_outputs/` for all `metrics.json` files, loads them into a list of run records, and generates nine plots grouped into three categories.

Learning curves (one subplot per dataset, one line per condition+backbone combination):

- `acc_curves.png` — validation accuracy vs. epoch. Shows whether training has converged and whether different conditions converge at different rates.
- `f1_curves.png` — macro F1 vs. epoch. The primary metric used for model selection; reveals overfitting when F1 peaks and then declines.
- `precision_curves.png` — macro precision vs. epoch. Indicates how often the model's positive predictions are correct.
- `recall_curves.png` — macro recall vs. epoch. Indicates how many true positives the model captures; particularly important for the rotten and formalin classes.
- `loss_curves.png` — training loss and validation loss vs. epoch on the same axes. Divergence between the two signals overfitting.

Aggregate comparisons (summarize best results across all runs):

- `metrics_summary.png` — grouped bar chart showing the best macro F1, precision, recall, and accuracy for each run, organized by dataset and condition. Useful for a quick overview of which configuration wins.
- `heatmap.png` — a condition × backbone grid where each cell shows the best F1 achieved. Empty cells correspond to experiments not yet run.

Error analysis (inspect what the model gets wrong):

- `confusion_matrix.png` — row-normalized confusion matrix for each completed run at the final epoch. Each row represents a true class; each column a predicted class. Values on the diagonal are correct classifications; off-diagonal values reveal systematic confusions (e.g. formalin predicted as fresh).
- `per_class.png` — bar chart of F1, precision, and recall broken down by individual class for each run. Identifies which specific class is pulling the macro average down.

All plots are saved to `run_outputs/_plots/`. Running `docs/prepare_figures.py` copies them to `docs/figures/` for inclusion in this report.